

Unidad de trabajo 1

Manejo de ficheros – Parte II

Acceso a datos

2025-2026

Técnico superior en Desarrollo de Aplicaciones Multiplataforma

Introducción

Durante el primer tema, hemos visto como manejar ficheros de texto mediante distintas clases y métodos que nos permiten manipularlos. Durante este segundo tema, trabajaremos con el manejo de ficheros binarios y de acceso aleatorio.

Estos ficheros toman más relevancia que los ficheros de texto en el uso de aplicaciones, por tanto serán más importantes para el programador. Recuerda que los ficheros binarios, están pensados para ser leídos por un programa, por lo que **es necesario conocer los tipos de dato** que contiene y su estructura.



Fichero binario

0	00000000	Un número entero: 14
1	00000000	
2	00000000	
3	00001110	
4	00000000	Otro número entero: 33
5	00000000	
6	00000000	
7	00100001	
...	...	

Fichero de texto

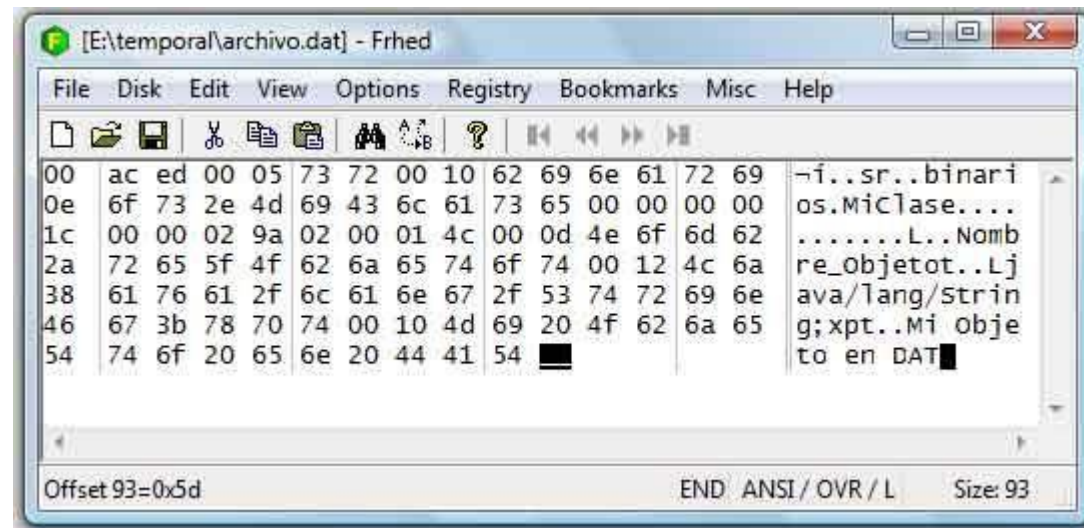
0	00110001	'I' (código ASCII 0x31)
1	00110100	'4' (código ASCII 0x34)
2	01101000	'h' (código ASCII 0x68)
3	01101111	'o' (código ASCII 0x6F)
4	01101100	'l' (código ASCII 0x6C)
5	01100001	'a' (código ASCII 0x61)
...	...	

Ficheros binarios

Los ficheros binarios almacenan secuencias de dígitos binarios que no son legibles directamente por el usuario como ocurría con los ficheros de texto.

Tienen la ventaja de que **ocupan menos espacio en disco.**

En Java las dos clases que nos permiten trabajar con este tipo de ficheros con **FileInputStream** y **FileOutputStream**, estas trabajan con flujos de bytes y crean un enlace entre el flujo de bytes y el fichero.



La case FileInputStream

Los métodos que proporciona la clase **FileInputStream** para lectura son similares a los vistos para la clase FileReader, estos métodos devuelven el número de bytes leídos.

Estos son los métodos que proporciona FileInputStream:

- **Int read():** este método lee un byte y lo devuelve
- **Int read(byte[] b):** este método lee una cantidad de bytes de datos correspondiente a b.length
- **Int read(byte[] b, int desplazamiento, int n):** este método lee hasta n bytes de la matriz b comenzando por desplazamiento y devuelve el número leído de bytes



La clase **FileOutputStream**

Esta clase **FileOutputStream**, también es muy similar, por lo que no nos resultará compleja su asociación.

Estos son los métodos que proporciona la clase **FileOutputStream**.

- **Int write():** este método escribe un byte
- **Int write(byte[] b):** este método escribe una cantidad de bytes de datos correspondiente a **b.length**
- **Int write(byte[] b, int desplazamiento, int n):** este método escribe hasta **n** bytes de la matriz **b** comenzando por **desplazamiento**.



Ejemplo lectura y escritura de ficheros binarios

En este ejemplo vamos a crear un fichero de datos .dat y básicamente lo llenaremos de bytes.

*Nota: El valor “true” del constructor del fichero de escritura indica que queremos añadir bytes al final.

```
import java.io.*;
public class FicherosBinariosEscribir {
    public static void main(String[] args) throws
        IOException{ File fichero = new
        File("FicheroPrueba.dat");
        FileOutputStream salida = new FileOutputStream(fichero, true);
        FileInputStream entrada = new FileInputStream(fichero);
        int i;
        for (i=0; i<100; i++){
            salida.write(i);
        }
        salida.close();
        while ((i=entrada.read()) != -1)
            System.out.println(i);
        entrada.close();
    }
}
```

Intenta visualizar el fichero de entrada que se ha generado con nuestro contenido, con un editor de texto



Las clases **DataInputStream** y **DataOutputStream**

Para leer y escribir datos primitivos (int, float, long, ...) usaremos las clases **DataInputStream** y **DataOutputStream**. Algunos de los métodos que implementan estas clases son:

PARA LECTURA

- Boolean readBoolean();
- Byte readByte()
- Int readUnsignedShort()
- Int readUnsignedByte()
- Short readShort()
- Char readChar()
- Int readInt()
- Long readLong()
- Float readFloat()
- Double readDouble()
- String readUTF()

.....

PARA ESCRITURA

- Boolean writeBoolean();
- Byte writeByte()
- Int writeUnsignedShort()
- Int writeUnsignedByte()
- Short writeShort()
- Char writeChar()
- Int writeInt()
- Long writeLong()
- Float writeFloat()
- Double writeDouble()
- String writeUTF()

.....



Ejemplo con DataOutputStream

```
import java.io.*;
public class FicherosBinariosArray {
    public static void main(String[] args) throws
        IOException{ File fichero = new
        File("FicheroArrays.dat");
        FileOutputStream salida = new FileOutputStream(fichero);
        DataOutputStream salidadatos = new DataOutputStream(salida);
        String marcas[] = {
            "Marca1","Marca2","Marca3","Marca4","Marca5","Marca6","Marca7",
            "Marca8","Marca9","Marca10","Marca11", "Marca12","Marca13","Marca14"
        };
        int anios[] = {2,4,5,1,2,2,9,12,2,5,7,9,2,2};
        };
        for (int i = 0; i < anios.length; i++)
        { salidadatos.writeUTF(marcas[i]);
          salidadatos.writeInt(anios[i]);
        }
        salidadatos.close();
    }
}
```

Vamos a ver un ejemplo en el que insertamos datos en un fichero a partir de dos arrays que contienen nombres y años respectivamente.



Ejemplo con DataInputStream

```
import java.io.*;
public class FicherosBinariosLeer {
    public static void main(String[] args) throws
        IOException{ File fichero = new
        File("FicheroArrays.dat");
        FileInputStream entrada = new FileInputStream(fichero);
        DataInputStream entradadatos = new DataInputStream(entrada);
        String marca;
        try {
            while (true) {
                marca = entradadatos.readUTF();
                anio =entradadatos.readInt();
                System.out.println("Marca: " + marca + " anio: " + anio);
            }
        } catch (EOFException e) {
            entradadatos.close();
        }
    }
}
```

Vamos a recuperar datos del fichero creado anteriormente. Estos se recuperan en el mismo orden en el que se han escrito, primero la marca y luego el año



Actividad 2.1



- a) Realiza un programa Java que cree un fichero binario para guardar datos de los alumnos, dale el nombre alumnos.dat.
- b) Introduce los siguientes datos por cada alumno (x5): Nia(entero), Nombre(String), Apellidos(String), Sexo (Char), Ciclo (String), Curso(String), Grupo(String).
- c) Realiza un programa Java que lea el fichero binario creado y pinte por pantalla el listado de alumnos.

Objetos serializable en ficheros binarios

En el ejercicio anterior, hemos visto que podemos guardar un alumno con varios atributos mediante datos primitivos, pero guardar estos datos por separado puede resultar engorroso, sobre todo si tenemos gran cantidad de objetos. Java nos permite guardar objetos en fichero binarios si estos objetos implementan la interfaz **Serializable**, la cual dispone de varios métodos para guardar y leer objetos en fichero binarios. Los más importante son:

- **Void readObject():** para leer un objeto.
- **Void writeObject(ObjectOutputStream stream):** para escribir un objeto.

La serialización de objetos de java, permite tomar cualquier objeto que implemente la interfaz **Serializable** y convertirlo en una secuencia de bits. Esta secuencia de bits puede además ser restaurada para regenerar el objeto original.



Las clases `ObjectInputStream` y `ObjectOutputStream`

Para leer y escribir objetos serializables a un stream, se utilizan las clases Java **`ObjectInputStream`** y **`ObjectOutputStream`**. Recuerda que nuestra clase tiene que implementar la **interfaz `Serializable`** para escribir o leer objeto en un fichero binario.

En un ejemplo, lo primero que vamos a hacer es crearnos nuestro objeto “Persona” que implementa la interfaz serializable, que implementa los métodos `et` y `get` para dar valor a sus atributos.

```
public class Persona implements Serializable{
    private String nombre;
    private int edad;

    public Persona (String nombre, int edad)
    { this.nombre = nombre;
      this.edad = edad;
    }
    public Persona ()
    { this.nombre =
      null;
    }

    public void setNombre (String nombre )
    { this.nombre = nombre;
    }
    public void setEdad (int edad)
    { this.edad = edad;
    }

    public String getNombre()
    { return this.nombre;
    }
    public int getEdad()
    { return
      this.edad;
    }
}
```



Escritura de objetos en ficheros binarios

Una vez disponemos de nuestro objeto. Utilizaremos el método **WriteObject()** para la su escritura en el fichero: `import java.io.*;`

```
public class EscribirFichObject {  
  
    public static void main(String[] args) throws IOException {  
        Persona persona;  
        File fichero = new File ("FichPersona.dat");  
        FileOutputStream ficheroSalida = new FileOutputStream (fichero);  
        ObjectOutputStream dataOS = new ObjectOutputStream (ficheroSalida);  
  
        String nombres[] = {"Ana", "Luis Miguel", "Alicia", "Pedro", "Manuel",  
                             "Andrés", "Julio", "Antonio", "Maria Jesús"};  
        int edades[] = {14,15,13,15,16,12,16,14,13};  
  
        for (int i=0; i<edades.length; i++){  
            persona = new Persona (nombres[i],edades[i]);  
            dataOS.writeObject(persona);  
        }  
        dataOS.close();  
    }  
}
```



Lectura de objetos en ficheros binarios

Utilizaremos el método `readObject()` para leer los objetos del flujo de entrada. Además de las excepciones ya vistas para la escritura, debemos tener en cuenta cuando lleguemos al final del fichero:

```
import java.io.*;
public class LeerFichObject {
    public static void main(String[] args) throws IOException, ClassNotFoundException {
        Persona persona;
        File fichero = new File ("FichPersona.dat");
        FileInputStream ficheroEntrada = new FileInputStream (fichero);
        ObjectInputStream dataIS = new ObjectInputStream (ficheroEntrada);
        try {
            while (true) {
                persona = (Persona) dataIS.readObject();
                System.out.printf("Nombre: %s, edad: %d %n", persona.getNombre(), persona.getEdad());
            }
        } catch (EOFException eo) {
            System.out.println("Fin de Lectura");
        }
        dataIS.close();
    }
}
```



Actividad 2.2

- a) Repite el ejercicio 2.1, pero esta vez utiliza un objeto Alumno.



Fichero de acceso aleatorio

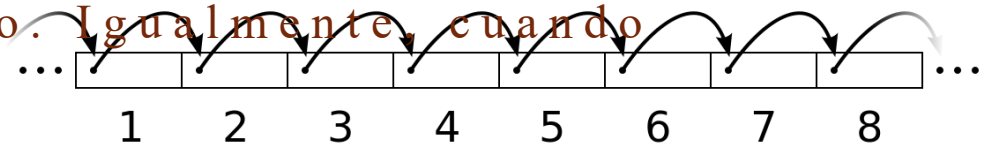


Hasta ahora todas las operaciones que hemos ido realizando sobre los ficheros se realizaban de forma secuencial.

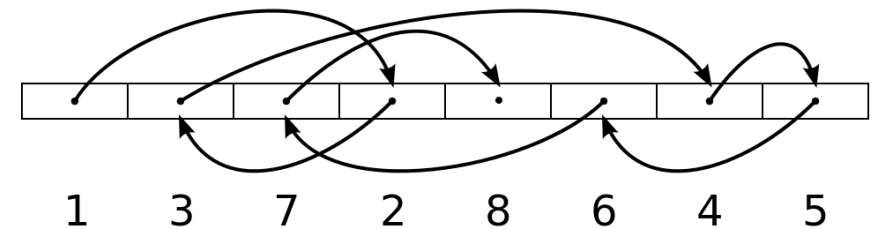
Se empezaba la lectura en el primer byte o el primer carácter o el primer objeto y seguidamente, se leían los siguientes uno tras de otro hasta llegar al fin del fichero. Igualmente, cuando escribíamos los datos en el fichero se iban escribiendo a continuación de la última información almacenada.

Java dispone de la clase **RandomAccessFile** que dispone de métodos para acceder al contenido de un fichero binario de forma aleatoria y para posicionarnos en un punto concreto del mismo.

Acceso secuencial



Acceso aleatorio



La clase RandomAccessFile

Es IMPORTANTE destacar que esta clase **no es parte de la jerarquía InputStream/OutputStream**, ya que su comportamiento es totalmente distinto puesto que se puede avanzar y retroceder dentro de un fichero.

Disponemos de dos constructores para crear el fichero de acceso aleatorio y ambos pueden lanzar la excepción FileNotFoundException:

- **RandomAccessFile (String nombreFichero, String modoAcceso):** Escribe el nombre del fichero incluido en el path
- **RandomAccessFile (File objetoFichero, String modoAcceso);** Utilizaría un objeto de tipo File asociado al fichero

El argumento modoAcceso puede tomar los siguientes dos valores:

- **“r”:** Abre el fichero en modo **sólo lectura**. El fichero debe existir
- **“rw”:** Abre el fichero en modo **lectura y escritura**. Si no existe el fichero se crea.



Uso de los flujos **DataInputStream** y **DataOutputStream**

Una vez abierto el fichero podemos usar la clases, ya vistas en el tema, **DataInputStream** y **DataOutputStream** utilizando sus métodos **read()** y **write()**.

En este caso, la clase **RandomAccessFile** maneja un **puntero** que indica la posición actual en el fichero. Cuando en el fichero se crea el puntero se coloca en 0, apuntando al principio del mismo. Las sucesivas llamadas a los metodos **read()** y **write()** ajustan el puntero según la cantidad de bytes leídos escritos.

Los métodos más importantes son los siguientes.

- **Long getFilePointer():** Devuelve la posición actual del puntero del fichero.
- **Void seek(long posicion):** Coloca el puntero del fichero en una posición determinada desde el comienzo del mismo.
- **Long length():** Devuelve el tamaño del fichero en bytes. La posición **length()** marca el final del fichero.
- **Int skipBytes(int desplazamiento):** Desplaza el puntero desde la posición actual el número de bytes indicados en desplazamiento.



```
import java.io.*;
```

Ejemplo de escritura de fichero de acceso aleatorio



```
    public class EscribirFicheroAleatorio {
        public static void main(String[] args) throws IOException{
            File fichero = new File ("FicheroAleatorio.dat");
            RandomAccessFile file = new RandomAccessFile(fichero, "rw");
            String apellido[] = { "FERNANDEZ", "GIL", "LOPEZ", "RAMOS", "SEVILLA", "CASILLA", "REY"};
            int dep[] = {10, 20, 10, 10, 30, 30, 20};
            Double salario[] = {1000.50, 2400.50, 2000.0, 1650.50, 950.0, 950.50, 2000.50};
            StringBuffer buffer = null;
            int n= apellido.length;
            for (int i = 0; i < n; i++) {
                file.writeInt(i+1);
                buffer = new StringBuffer(apellido[i]);
                buffer.setLength(10); // 10 caracteres del apellido
                file.writeChars(buffer.toString()); // escribimos el apellido
                file.writeInt(dep[i]);
                file.writeDouble(salario[i]);
            }
            file.close();
        }
    }
```

Insertamos secuencialmente los datos: apellido, departamento, y salario. Cada empleado tendrá un identificador (i+1) y la longitud de cada registro de empleados será de 36 bytes (*)

Debemos abrir el fichero en modo “rw”

(*)Longitud de los registros

Para calcular los 36 bytes de cada registro de empleados, se ha tenido en cuenta:

- Se inserta en primer lugar un entero, como identificador, **4 bytes**
- Después una cadena de 10 caracteres. Hay que tener en cuenta que Java utiliza caracteres UNICODE, por tanto $1 \text{ Carácter} = 16 \text{ bits} = 2 \text{ bytes} \times 10 \text{ Caracteres} = \mathbf{20 \text{ Bytes}}$
- Un tipo entero que es el departamento, ocupa **4bytes**
- El salario de tipo doublé ocupa **8 bytes**

Otros tipos:

Short (2 bytes), byte (1 byte), long (8 bytes), boolean (1 bit), float(4 bytes),.....



```
import java.io.*;
```

```
public class LeerFicheroAleatorio {
```

```
    public static void main(String[] args) throws
```

```
        IOException{ File fichero = new
```

```
        File("FicheroAleatorio.dat");
```

```
        RandomAccessFile file = new RandomAccessFile(fichero, "r");
```

```
        int id, dep, posicion;
```

```
        Double salario;
```

```
        char apellidos[] = new char[10], aux;
```

```
        for(;;){
```

```
            file.seek(posicion);
```

```
            id=file.readInt();
```

```
            for (int i = 0; i < apellidos.length; i++)
```

```
                { aux = file.readChar();
```

```
                  apellidos[i] = aux;
```

```
            }
```

```
            String apellidoX = new String(apellidos);
```

```
            dep=file.readInt();
```

```
            salario=file.readDouble();
```

```
            System.out.println("Identificador: " + id + " Apellido: " + apellidoX + " Departamento: " + dep + "Salario: " + salario);
```

```
            posicion = posicion + 36;
```

```
            if (file.getFilePointer() ==
```

```
                file.length())break;
```

```
        }
```

```
        file.close();
```

```
    }
```

```
}
```

Ejemplo de lectura de fichero de acceso aleatorio



Para recuperar un empleado nuevo, hay que sumar el tamaño del registro (36) a la variable utilizada para el posicionamiento

Cuestión



¿Cómo accederías a un registro concreto, suponiendo que conocemos el identificador?

¿Qué debería hacer para escribir un nuevo registro al final de mi fichero?



Respuesta



¿Cómo accederías a un registro concreto, suponiendo que conocemos el identificador?

- A partir del identificador debemos calcular la posición a partir del tamaño del registro
 - **Posición = (identificador – 1) * 36**

¿Qué debería hacer para escribir un nuevo registro al final de mi fichero?

- Debemos posicionar el puntero al final de este fichero para comenzar a escribir.
 - **Long posición= file.length();**
 - **File.seek (posición);**



Actividad 2.3

- a) Crea un programa Java que consulte los datos de un empleado de nuestro fichero. Este programa debe pedir al usuario el identificador por línea de comandos. Si el empleado existe, visualizaremos los datos, si no existe se mostrará un mensaje.
- b) Crea un programa Java que inserte datos en el fichero aleatorio. Se debe pedir al usuario que introduzca los datos por pantalla y nuestro programa debe verificar si existe el id introducido antes de insertar, si este id ya existe mostraremos un mensaje.
- c) Crea un programa Java que permita modificar el salario de un empleado. Para ello recibirá por pantalla un identificador y un importe nuevo. Si el id no existe mostrará un mensaje y en caso de que exista mostrará el apellido y el nuevo salario.
- d) Crea un programa que permita el borrado lógico de un empleado. Para ello solicitaremos el id del empleado y guardaremos su id como -1
- e) Crea un programa que recorra el fichero y muestre los apellidos de lo empleados borrados



¿Preguntas?